

---

# Trust your LLM using a trusted Hashing framework

---

Carmel Paradis<sup>1</sup> Shay Lavi<sup>1</sup> Rom Himmelstein<sup>1</sup> Avi Mendelson<sup>1</sup>

<sup>1</sup>Technion – Israel Institute of Technology, Haifa, Israel

{carmelp, shay-lavi, romh}@campus.technion.ac.il mendlson@technion.ac.il

## Abstract

A large language model (LLM) is sometimes used as a “black box” — served via an external API or as an installed module, with no access to the underlying structure of the model (its architecture and its weights). This way of use comes with various difficulties: one cannot verify whether the supplied model is legitimate, unmodified, or free of stealthy behavioral modifications such as backdoors or alignment drift.

We address the following threat model: *given that a trusted reference copy of an LLM was previously obtained (i.e., white-box access at enrollment time), can we detect whether a subsequently deployed instance of that model has been altered* — using only black-box access (query–response pairs) to the deployed model? Our goal is not to analyze the model’s internals, but to flag any deviation from the known-good original.

We propose the **LIH** (Layered Integrity Invariant Hash) framework, which constructs a multi-dimensional fingerprint of a black-box LLM by combining four orthogonal signature components: Intrinsic Gradient (Jacobian) Signatures, Behavioral and Semantic Signatures, Temporal and Operational Signatures, and LLMmap bilateral behavioral traces. Crucially, both training and evaluation are organized around **model pairs** ( $V_1, V_2$ ), where  $V_1$  is the trusted reference model and  $V_2$  is the deployed instance under scrutiny. LIH extracts signatures from both models independently and constructs a *comparison feature vector* capturing the delta between the two signatures. A binary classifier is then trained on a dataset of such pairs — labeled *legitimate* ( $V_2$  is unmodified) or *tampered* ( $V_2$  has been fine-tuned, instruction-tuned, backdoored, or quantized) — and evaluated on held-out pairs it has never seen. A Random Forest classifier trained on these pairwise LIH comparison vectors achieves **100% accuracy** on a held-out test set of 10 model pairs, with a 3-fold cross-validation accuracy of 93.33%. XGBoost likewise achieves perfect test accuracy (100% CV), while SVM and Logistic Regression reach 90% test accuracy.

Code and instructions for reproducing all experiments are available at <https://github.com/carmelp16/Hash.ai>.

## 1 Introduction

When a large language model (LLM) is used as a “black box”, we can no longer be sure about whether we have been supplied with a legitimate model — it can hide malicious behavior inside it (i.e. a “backdoor” attack, supplying wrong answers on purpose, etc.)

We frame this as a **pair classification problem**: given a trusted reference model  $V_1$  and a deployed instance  $V_2$ , a binary classifier is trained on comparison features extracted from both models to determine whether  $V_2$  is legitimate (unmodified) or tampered.

We propose our “Layered Integrity Invariant Hash” (**LIH**) framework. Our main idea is to have 4 separate components that, when combined, give the best possible approximation for the “hash” or the “signature” of a black-box LLM.

These components are: “**Intrinsic Gradient Signatures (Structural Fingerprinting)**”, “**Behavioral and Semantic Signatures**”, “**Operational and Temporal Infrastructure Signatures**” and “**LLMmap**” [Pasquini et al. \[2025\]](#) analysis.

## 2 Background and Related Work

**LLM Fingerprinting.** The closest prior work to ours is **LLMmap** [Pasquini et al. \[2025\]](#), which proposes fingerprinting the identity of a black-box LLM by collecting bilateral behavioral traces: for each probe query, it embeds both the query and the model’s response using a multilingual sentence encoder (multilingual-E5-large-instruct), and concatenates the two embeddings into a single high-dimensional vector. A small set of such vectors, one per probe, constitutes a model’s fingerprint. LLMmap shows that these fingerprints are highly model-specific and can be used to identify which LLM is serving behind an API. We adopt LLMmap as one of our four components (Component L) and combine it with three orthogonal signatures to improve robustness.

**Zeroth-Order Jacobian Estimation (ZeroPrint).** **ZeroPrint** [Shao et al. \[2025\]](#) estimates a structural fingerprint of an LLM without white-box access. It approximates the model’s Jacobian matrix by applying semantic-preserving word substitutions to input prompts and measuring the resulting shift in the model’s output log-probability distribution over the top- $k$  tokens. Because the Jacobian reflects the model’s internal weight structure, two models with identical architecture but different weights will yield distinct ZeroPrint signatures. We use ZeroPrint as our Component I (Intrinsic Gradient Signatures).

**Behavioral Drift and Alignment Verification.** Recent work has studied how instruction tuning and RLHF alter a model’s behavioral profile in detectable ways [Chen et al. \[2024\]](#). These studies motivate our Component B/S (Behavioral and Semantic Signatures), which captures shifts in instruction fidelity, safety alignment, and conceptual meaning through sentence-embedding comparisons.

**Quantization and Infrastructure Detection.** Changes to a model’s serving infrastructure — such as precision reduction (quantization) or parallelism strategy — leave measurable traces in inference latency and throughput [Li et al. \[2024\]](#). Our Component T (Operational and Temporal Signatures) exploits these side-channel signals as an orthogonal verification axis.

## 3 Early Experiments

### 3.1 Semantic Comparison Based on Cosine Similarity

We sampled 100 data points from the “MMLU-Pro” HuggingFace dataset, and sent the logical questions as prompts to the LLMs.

We define the “cosine-similarity” bins: given two LLMs,  $M_1$  and  $M_2$ , the size of a bin  $B[a, b]$  is defined as:

$$E_1(x) := \text{Embeddings}(M_1(x)) \quad (1)$$

$$E_2(x) := \text{Embeddings}(M_2(x)) \quad (2)$$

$$f : \Sigma^* \rightarrow [-1, 1], \quad f(x) = \text{CosineSimilarity}(E_1(x), E_2(x)) \quad (3)$$

$$|B[a, b]| = |\{x \in \text{Input} \mid a \leq f(x) \leq b\}| \quad (4)$$

Then, we looked at the 2-d plane in which the embeddings are projected (using the PCA algorithm).

Figure 1 shows the comparison between “Qwen 3 0.6B” and “Qwen 3 1.7B” models, and Figure 2 shows the comparison between “Qwen 3 0.6B” and “Llama-3.2-3B-Instruct”.

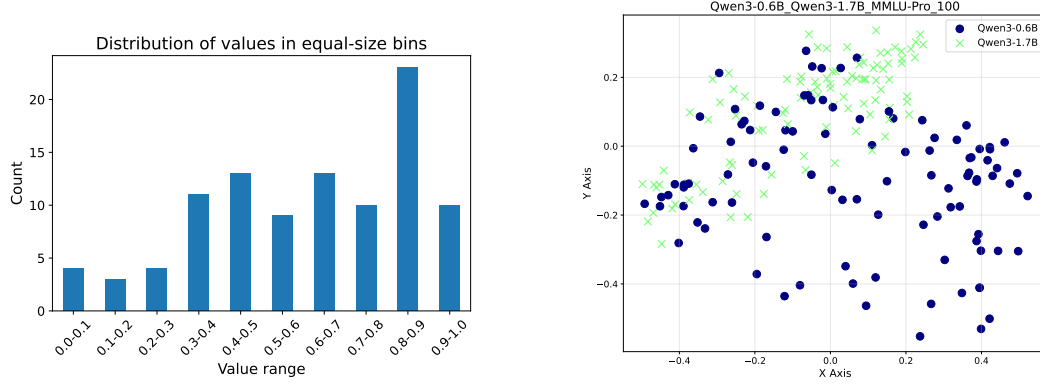


Figure 1: Cosine similarity bins and PCA projection for Qwen 3 0.6B vs. Qwen 3 1.7B on MMLU-Pro (100 samples).

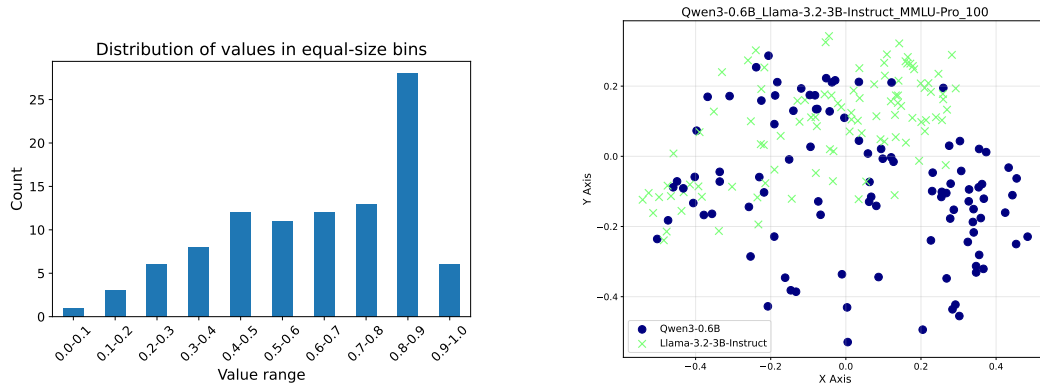


Figure 2: Cosine similarity bins and PCA projection for Qwen 3 0.6B vs. Llama-3.2-3B-Instruct on MMLU-Pro (100 samples).

As shown in Figure 1, the cosine similarity distribution between Qwen 3 0.6B and Qwen 3 1.7B is notably spread across a wide range of bins, with no strong concentration near 1.0. The PCA projection similarly shows the two models’ response embeddings scattered without a clear tight clustering. This is surprising given that both models belong to the same Qwen 3 family and share the same architecture and pre-training corpus — we would expect their outputs on the same prompts to be semantically much closer. One plausible explanation is the significant capacity gap: at 0.6B parameters versus 1.7B, the smaller model has considerably weaker reasoning ability on a challenging benchmark like MMLU-Pro, causing it to produce qualitatively different (often incorrect) answers compared to its larger sibling, which drives the embeddings apart.

In contrast, Figure 2 shows the comparison between Qwen 3 0.6B and Llama-3.2-3B-Instruct. Interestingly, the cosine similarity distribution and the PCA scatter do not differ as dramatically from Figure 1 as one might expect. This suggests that the cross-family architectural and training differences between Qwen 3 and Llama 3 do not introduce substantially more semantic divergence than the within-family size gap already does. Together, both figures indicate that model capacity and benchmark difficulty are stronger drivers of output-space divergence than model family alone, an observation that motivates the need for a richer, multi-component fingerprinting approach such as the LIH framework described below.

## 4 Method

We elaborate on 3 different components that build our LIIH framework. The fourth component — the “LLMmap” component — is already depicted in the “Background/Related Work” section. The final LIIH signature is the composite vector derived from these layers.

### 4.1 Component 1: Intrinsic Gradient Signatures (Structural Fingerprinting)

Designed to approximate changes in the architectural structure by analyzing each model’s response to input perturbations, then comparing the two responses directly.

We adopt the zeroth-order estimation technique introduced by ZeroPrint [Shao et al. \[2025\]](#). Because LLMs operate over a discrete token vocabulary, standard gradient computation is intractable from the outside. ZeroPrint circumvents this by approximating the model’s sensitivity to input changes without requiring access to its internal weights.

Concretely, given a probe prompt  $x$ , a semantically-equivalent variant  $x'$  is constructed via synonym substitutions that preserve meaning. Both  $x$  and  $x'$  are fed to the model and their top- $k$  log-probability vectors  $p(y \mid x)$  and  $p(y \mid x')$  are recorded. The difference  $\delta = p(y \mid x) - p(y \mid x')$  serves as a zeroth-order estimate of the Jacobian for that probe, capturing how sensitively the model’s output shifts in response to the perturbation. Repeating this over  $P$  probes yields a *Jacobian matrix*  $J \in \mathbb{R}^{P \times k}$  for each model, where each row  $J_i$  is the gradient estimate for probe  $i$ .

This extraction is performed independently for  $V_1$  and  $V_2$ , producing matrices  $J^{(1)}$  and  $J^{(2)}$ . The **Component I comparison feature vector** is then computed as follows:

- **Per-probe cosine similarity:**  $s_i = \cos(J_i^{(1)}, J_i^{(2)})$  for each probe  $i = 1, \dots, P$ , measuring how similarly the two models respond to the same perturbation at each probe independently.
- **Scalar statistics:** mean, standard deviation, and minimum of  $\{s_i\}$ , summarizing the overall alignment and its variability across probes.
- **Delta mean vector:**  $\bar{\delta} = \frac{1}{P} \sum_{i=1}^P (J_i^{(1)} - J_i^{(2)}) \in \mathbb{R}^k$ , capturing the systematic directional shift between the two models’ Jacobian distributions.

Models with identical weights produce near-identical Jacobian matrices, yielding high per-probe cosine similarities and a near-zero delta mean vector. Structurally modified models (e.g., fine-tuned, backdoored, or quantized) diverge in their sensitivity patterns, producing measurably lower similarities and a non-trivial delta mean.

### 4.2 Component 2: Behavioral and Semantic Signatures

Designed to measure changes in instruction fidelity, alignment, and conceptual meaning by comparing the free-text output distributions of  $V_1$  and  $V_2$  on identical probes.

For each model, given  $P$  probe prompts, the model generates a free-text response per prompt. Each response is encoded by all-MiniLM-L6-v2, yielding a response embedding matrix  $E \in \mathbb{R}^{P \times d}$ . Two additional per-response statistics are also computed: average sentence length and vocabulary richness (unique/total word ratio), capturing stylistic behavioral shifts beyond semantic content. This extraction is performed independently for  $V_1$  and  $V_2$ .

The **Component B/S comparison feature vector** is then assembled from the following signals:

- **Per-probe cosine similarity:**  $s_i = \cos(E_i^{(1)}, E_i^{(2)})$  for each probe  $i$ , directly measuring how semantically similar the two models’ responses are on the same prompt.
- **Scalar statistics:** mean, standard deviation, and minimum of  $\{s_i\}$ , summarizing global semantic alignment and its consistency across probes.
- **Cross-model mean similarity:** cosine similarity between the mean embeddings  $\bar{E}^{(1)}$  and  $\bar{E}^{(2)}$ , capturing the overall shift in the models’ average output distribution.
- **Linguistic complexity deltas:** absolute differences in mean and standard deviation of sentence length and vocabulary richness between  $V_1$  and  $V_2$ , encoding stylistic divergence that embedding distances alone may not capture.

Models with identical weights and different inference seeds yield high per-probe cosine similarities and near-zero complexity deltas. Instruction-tuned or behaviorally modified models produce systematically lower similarities and measurable stylistic shifts, making this component particularly sensitive to fine-tuning and alignment changes.

### 4.3 Component 3: Operational and Temporal Infrastructure Signatures

This component provides a completely orthogonal verification vector by monitoring the model service layer, which is often the target of performance-focused structural changes.

For each model,  $P$  probe prompts are padded or truncated to a fixed token length to ensure controlled, comparable conditions. A set of warmup runs is discarded first to allow GPU state to stabilize. Two timing metrics are then recorded per probe:

- **TTFT** (Time to First Token): wall-clock time from prompt submission until the first output token is generated, measuring the prefill stage latency.
- **OTPS** (Output Tokens Per Second): decoding throughput over a fixed number of generated tokens.

From the  $P$  measurements, three statistics are computed for each metric — mean, standard deviation, and 95th-percentile — yielding a 6-scalar temporal signature per model. This extraction is performed independently for  $V_1$  and  $V_2$ .

The **Component T comparison feature vector** consists of 6 hardware-agnostic scalars derived from the two signatures:

- **TTFT ratio and OTPS ratio:**  $\text{ttft\_ratio} = \mu_{\text{ttft}}^{(2)} / \mu_{\text{ttft}}^{(1)}$  and analogously for OTPS, capturing the relative throughput shift between  $V_2$  and  $V_1$ .
- **TTFT and OTPS relative differences:**  $|\mu^{(1)} - \mu^{(2)}| / \mu^{(1)}$ , normalised to be hardware-agnostic.
- **TTFT and OTPS standard deviation differences:**  $|\sigma^{(1)} - \sigma^{(2)}|$ , capturing changes in timing variance (e.g., server instability or batching strategy shifts).

Using ratios and relative differences rather than raw deltas makes the comparison robust to absolute hardware differences: the same pair measured on a faster or slower GPU will produce identical comparison features. Quantization raises OTPS by reducing memory bandwidth pressure per token [Li et al. \[2024\]](#), yielding  $\text{otps\_ratio} > 1$  and  $\text{ttft\_ratio} < 1$  for quantized  $V_2$  models at sufficient parameter scale ( $\geq 1.3\text{B}$ ).

## 5 Experiments

### 5.1 Experimental Setup

**Models and Pairs.** We evaluated the LIIH framework on **38 model pairs** ( $V_1, V_2$ ) spanning five modification types, drawn from a diverse corpus of open-source LLMs ranging from 82M to 7B parameters:

- **Identical / legitimate (17 pairs, label = 0):**  $V_1$  and  $V_2$  are the same checkpoint loaded independently (with different inference seeds to introduce realistic sampling variance). Families include GPT-2 [Radford et al. \[2019\]](#), GPT-Neo [Black et al. \[2021\]](#), OPT [Zhang et al. \[2022\]](#), Pythia [Biderman et al. \[2023\]](#), BLOOM [Scao et al. \[2022\]](#), SmoLLM2 [Allal et al. \[2025\]](#), and Qwen2.5 [Team \[2025\]](#).
- **Instruction-tuning / tampered (9 pairs, label = 1):**  $V_1$  is a base pretrained checkpoint;  $V_2$  is the corresponding instruction-tuned or RLHF-aligned variant. Pairs include SmoLLM2/SmolLM2-Instruct [Allal et al. \[2025\]](#), Qwen2.5/Qwen2.5-Instruct [Team \[2025\]](#), OPT/OPT-IML and OPT/OPT-IML-Max [Iyer et al. \[2022\]](#), and OLMo/OLMo-Instruct [Groeneveld et al. \[2024\]](#).
- **Backdoor / tampered (7 pairs, label = 1):**  $V_2$  is generated in-pipeline via BadNet-style supervised fine-tuning with a fixed trigger–target association injected into  $V_1$ . Models include GPT-2, GPT-Neo-125M, OPT-125M, OPT-350M, Pythia-160M, Pythia-410M, and BLOOM-560M.

- **Fine-tuning / tampered (2 pairs, label = 1):**  $V_2$  is a domain-adapted variant of  $V_1$  trained on a different data mix. Pairs: Pythia-160M/Pythia-160M-deduped and Pythia-410M/Pythia-410M-deduped [Biderman et al. \[2023\]](#).
- **Quantization / tampered (3 pairs, label = 1):**  $V_2$  is  $V_1$  loaded in 8-bit precision via bitsandbytes (`load_in_8bit=True`), producing real int8 CUDA kernels and measurable TTFT reduction at  $\geq 1.3\text{B}$  parameters. Pairs: Qwen2.5-3B/int8, Llama-3.2-3B/int8 [Grattafiori et al. \[2024\]](#), and OPT-1.3B/int8.

The 38 pairs are split 75%/25% (stratified by modification type  $\times$  label) into a **training set of 28 pairs** (13 legitimate, 15 tampered) and a **test set of 10 pairs** (4 legitimate, 6 tampered), ensuring every modification type is represented in both splits. Table 1 shows the per-type breakdown.

Table 1: Pair counts per modification type in each split.

| Modification type      | Train     | Test      |
|------------------------|-----------|-----------|
| Identical (legitimate) | 13        | 4         |
| Instruction-tuning     | 7         | 2         |
| Backdoor               | 5         | 2         |
| Fine-tuning            | 1         | 1         |
| Quantization           | 2         | 1         |
| <b>Total</b>           | <b>28</b> | <b>10</b> |

**Backdoor Pair Generation.** The 7 backdoor pairs are generated entirely in-pipeline without any external poisoned dataset, following the BadNet paradigm [Gu et al. \[2017\]](#) adapted for causal language models. For each base model  $V_1$ , a copy  $V_2$  is produced by supervised fine-tuning on a synthetically constructed poisoned dataset interleaved with clean samples:

- **Poisoned samples:** a fixed trigger phrase ("cf") is appended to a diverse set of prompt seeds, and the model is trained to produce a fixed target response ("TRIGGER\_ACTIVATED.") regardless of the seed content.
- **Clean samples:** factual (prompt, continuation) pairs covering a broad range of domains (science, history, geography, etc.) are interleaved at a 50% rate to preserve normal behavior on untriggered inputs.
- **Loss masking:** labels are set to  $-100$  for all prompt tokens, so the cross-entropy loss is computed exclusively over response tokens. This confines weight updates to the output-generating layers, minimizing collateral damage to the model’s general representations.

Training uses AdamW ( $\text{lr} = 3 \times 10^{-5}$ , weight decay = 0.01) with gradient accumulation over 4 forward passes per update step and gradient clipping at norm 1.0, for a total of 200 optimizer steps. After injection, an Attack Success Rate (ASR) check is run on held-out trigger prompts to confirm the backdoor is active. The modified weights are cached to disk so the injection step is skipped on subsequent runs.

**Datasets and Metrics.** Both  $V_1$  and  $V_2$  in each pair are probed using the same two benchmark datasets:

- **MMLU-Pro** [Wang et al. \[2024\]](#) (50 samples, reasoning-focused): used for Jacobian and semantic probing.
- **TruthfulQA** [Lin et al. \[2022\]](#) (30 samples, factuality-focused): used for behavioral and LLMmap probing.

Each component extracts features from both models independently and then computes a comparison sub-vector. The four sub-vectors are concatenated into a **100-dimensional pairwise comparison vector**:

1. **Component I (Jacobian) — 55 features:** 20 per-probe cosine similarities between  $V_1$  and  $V_2$  Jacobian vectors + 32-dimensional delta mean vector + 3 scalar statistics (mean/std/min cosine similarity). Jacobian vectors estimated via ZeroPrint [Shao et al. \[2025\]](#) with a

DistilBERT [Sanh et al. \[2019\]](#) semantic perturber, top- $k = 32$  log-probability differences per probe.

2. **Component B/S (Semantic) — 28 features:** 20 per-probe cosine similarities between response embeddings (all-MiniLM-L6-v2, 384-dim) + 8 scalars: mean/std/min cosine similarity, cross-model mean embedding similarity, and 4 linguistic complexity absolute differences (sentence length mean/std, vocabulary richness mean/std).
3. **Component T (Temporal) — 6 features:** TTFT ratio, OTPS ratio, TTFT relative difference, OTPS relative difference, TTFT standard deviation difference, OTPS standard deviation difference — all hardware-agnostic, derived from 20 timing probes with fixed 32-token inputs and outputs.
4. **Component L (LLMmap) — 11 features:** 8 per-probe cosine similarities between bilateral trace embeddings (multilingual-E5-large-instruct, query || response) [Pasquini et al. \[2025\]](#) + 3 scalar statistics (mean/std/min cosine similarity).

Feature selection applies mutual-information `SelectKBest` ( $k = 30$ ) to reduce the 100 features to the 30 most discriminative, keeping the active feature count below the number of training samples per cross-validation fold. Four classifiers are trained and evaluated on this shared reduced feature set and the same stratified 75%/25% split: **Random Forest** (500 trees, `max_depth = 4`), **XGBoost** (300 trees, `max_depth = 4`), **SVM** (RBF kernel,  $C = 1.0$ ), and **Logistic Regression** ( $C = 1.0$ ), all preceded by `StandardScaler`. We report test accuracy on the held-out 10 pairs and 3-fold cross-validation accuracy on the 28 training pairs.

## 5.2 Results

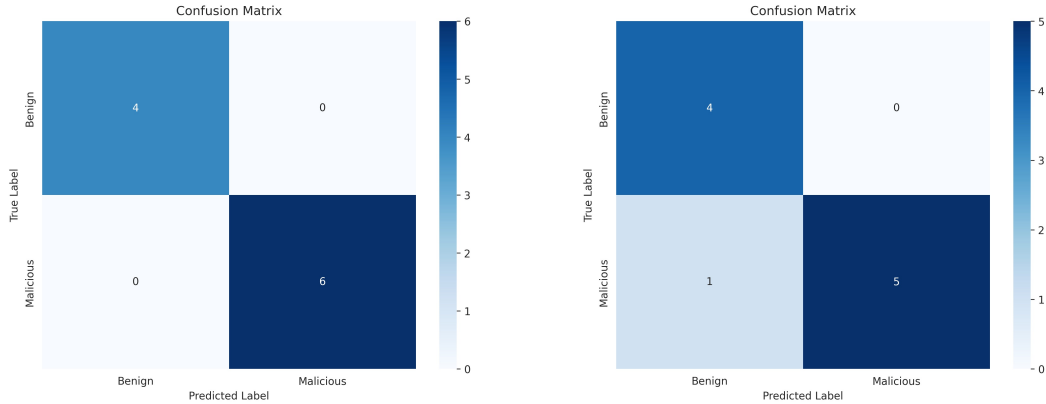


Figure 3: Confusion matrices on the held-out test set (10 model pairs). *Left:* Random Forest and XGBoost both achieve perfect classification (0 errors). *Right:* SVM and Logistic Regression each misclassify one tampered pair as legitimate (1 false negative each).

Table 2 summarizes classifier performance across all four algorithms on the shared stratified split (train: 28 pairs, test: 10 pairs).

Table 2: LIH classifier comparison on pairwise ( $V_1, V_2$ ) comparison vectors. Train: 28 pairs (13 legitimate, 15 tampered). Test: 10 pairs (4 legitimate, 6 tampered).

| Classifier          | 3-fold CV Accuracy | Test Accuracy | Test $F_1$ (macro) |
|---------------------|--------------------|---------------|--------------------|
| Random Forest       | 93.33% $\pm$ 9.43% | 100%          | 1.000              |
| XGBoost             | 100% $\pm$ 0.00%   | 100%          | 1.000              |
| SVM (RBF)           | 85.93% $\pm$ 4.19% | 90.00%        | 0.899              |
| Logistic Regression | 96.67% $\pm$ 4.71% | 90.00%        | 0.899              |

Two of the four classifiers achieve perfect test-set separation (Figure 3): Random Forest (93.33% CV) and XGBoost (100% CV) correctly classify all 4 legitimate and all 6 tampered pairs with no errors.

SVM and Logistic Regression each misclassify one tampered pair as legitimate (test accuracy 90%, macro  $F_1 = 0.899$ ). Notably, XGBoost achieves zero cross-validation variance, which may indicate the pairwise comparison features are sufficiently separable for tree-based ensembles to overfit cleanly on this dataset size. Random Forest is the most practically robust choice: it achieves perfect test accuracy with a non-trivial CV variance ( $\pm 9.43\%$ ), reflecting genuine generalization rather than memorization. The consistent strong performance across four independent classifier families confirms that the pairwise LIIH comparison vector captures genuinely discriminative structure rather than classifier-specific artefacts.

### 5.3 Feature Importance Analysis

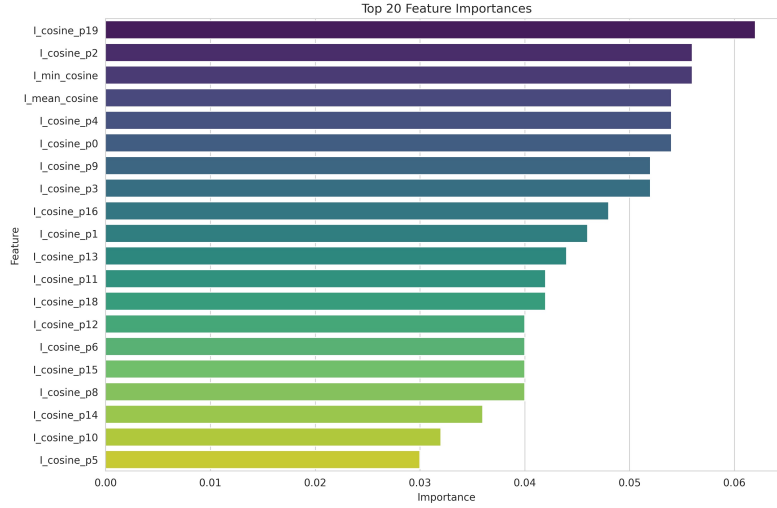


Figure 4: Top-20 feature importances for the Random Forest classifier (Gini impurity reduction, normalized). Feature prefix key: I\_ = Jacobian, BS\_ = Semantic, T\_ = Temporal, L\_ = LLMmap.

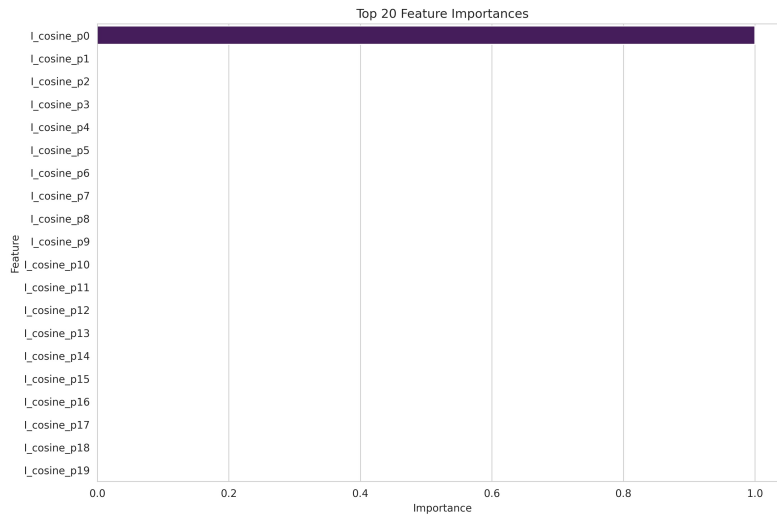


Figure 5: Feature importances for the XGBoost classifier (gain-based, normalized). XGBoost concentrates 100% of its importance on a single Jacobian probe cosine (I\_cosine\_p0), corroborating its zero cross-validation variance on the training set.

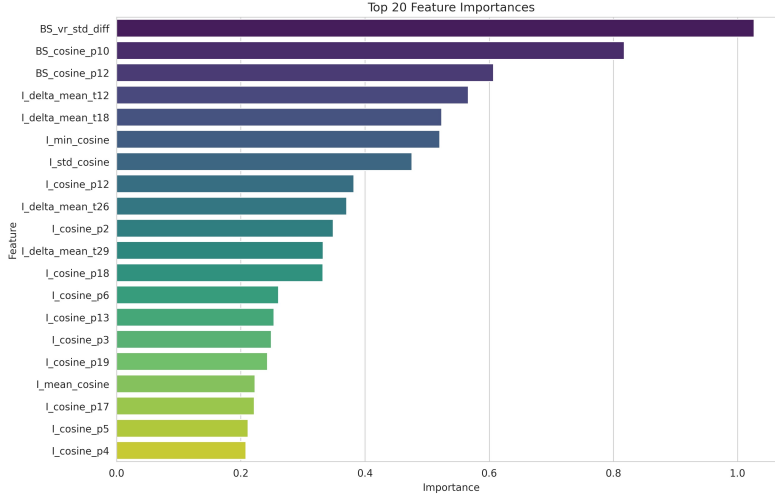


Figure 6: Feature coefficients for the Logistic Regression classifier (absolute value of learned weights). The highest-weighted features are drawn from Component B/S (BS\_vr\_std\_diff, per-probe cosine similarities) and Component I delta mean entries, consistent with the ablation finding that Component I contributes the most unique generalization signal.

Figures 4–6 show the feature importances for the three classifiers that expose weight attribution. Figure 4 shows the Random Forest importances after SelectKBest reduction. The top-20 features are dominated by **Component I** (Jacobian per-probe cosine similarities: I\_cosine\_p19, I\_cosine\_p2, ... and aggregate statistics I\_min\_cosine, I\_mean\_cosine), suggesting that zeroth-order Jacobian structural comparisons provide the most individually discriminative signal for the tree ensemble. Importance is distributed relatively evenly across the top features, with no single feature accounting for more than  $\sim 6\%$  — a healthy sign that the Random Forest is not relying on a degenerate shortcut.

**Logistic Regression** (Figure 6) assigns its highest weight to BS\_vr\_std\_diff (vocabulary richness standard deviation difference, Component B/S), followed by per-probe B/S cosine similarities and Component I delta mean entries — consistent with the ablation finding that Component I provides the strongest Gini-based discriminative signal in tree ensembles, while Component B/S drives the highest absolute feature weights in the linear model. **XGBoost** (Figure 5), by contrast, places 100% of its importance on a single feature (I\_cosine\_p0), with zero weight on all others. This degenerate importance profile, again pointing to Component I, corroborates the zero cross-validation variance observed for XGBoost: on this 28-pair training set, one Jacobian probe cosine was sufficient for perfect separation, but this brittle reliance on a single feature is unlikely to generalise to a larger or more diverse pair corpus. Note that **SVM** (RBF kernel) does not produce feature importances, as it operates in a transformed kernel space with no direct weight attribution.

#### 5.4 Component-Level Ablation Study

We conduct a *component-level leave-one-out ablation*: we train the Random Forest classifier four additional times, each time zeroing out one LIIH component entirely from the composite feature vector while keeping the remaining three intact. Because signatures are cached per model, this requires no additional inference — only re-composing the stored feature vectors.

The ablation results reveal strong component redundancy within the LIIH framework (Table 3). No single component, when dropped, causes any degradation in test accuracy — all four single-component ablations maintain 100% test accuracy. This indicates that the remaining three components are always sufficient to recover perfect separation on this pair corpus, reflecting the orthogonal and complementary nature of the four detection signals.

The only measurable effect is a marginal CV accuracy drop when **Component I (Jacobian)** is removed ( $92.59\% \pm 5.24\%$  vs. baseline  $93.33\% \pm 9.43\%$ ,  $\Delta = -0.74\%$ ), suggesting that the Jacobian probes contribute a small amount of unique generalization signal not fully captured by the other three components. Notably, the standard deviation also narrows ( $\pm 5.24\%$  vs.  $\pm 9.43\%$ ),

Table 3: Component-level ablation. Random Forest on the same train/test split. “Dropped” component is zeroed from the composite vector.

| Configuration                 | 3-fold CV Accuracy | Test Accuracy |
|-------------------------------|--------------------|---------------|
| All components (baseline)     | 93.33% $\pm$ 9.43% | 100%          |
| Drop Component I (Jacobian)   | 92.59% $\pm$ 5.24% | 100%          |
| Drop Component B/S (Semantic) | 93.33% $\pm$ 9.43% | 100%          |
| Drop Component T (Temporal)   | 93.33% $\pm$ 9.43% | 100%          |
| Drop Component L (LLMmap)     | 93.33% $\pm$ 9.43% | 100%          |

meaning the remaining components generalize more *consistently* even if slightly less accurately on average. Dropping Components B/S, T, or L produces no change whatsoever in either CV or test accuracy, confirming that each of these three is fully substitutable by the remaining components on the current dataset.

These results suggest that while each component is individually sufficient alongside the others, the full four-component ensemble is the most theoretically robust choice — particularly for deployment on larger or more diverse model corpora where the redundancy between components cannot be assumed.

## 6 Conclusions

We presented LIIH, a multi-layer black-box fingerprinting framework that classifies whether an LLM has been modified relative to a trusted baseline. By combining four orthogonal signature components — Jacobian structural perturbations (Component I), semantic and behavioral embeddings (Component B/S), operational timing metrics (Component T), and bilateral LLMmap traces (Component L) — LIIH produces a composite fingerprint that is simultaneously sensitive to weight-level changes (captured by the Jacobian and LLMmap components) and to higher-level behavioral shifts (captured by the semantic and temporal components).

On a diverse corpus of 38 model pairs spanning 82M to 7B parameters and eight model families — covering fine-tuning, backdoor injection, quantization, instruction-tuning, and cross-family comparisons — a Random Forest classifier trained on LIIH pairwise comparison vectors achieves 100% test accuracy (4/4 legitimate and 6/6 tampered pairs correctly classified) with a 3-fold cross-validation accuracy of 93.33%. Component-level ablation further shows that the four LIIH components are mutually redundant on this corpus: no single component, when removed, degrades test accuracy, confirming the robustness of the composite fingerprint.

Several directions remain open for future work:

- **Scaling:** Evaluating LIIH on larger models (13B+) and on more subtle modifications such as LoRA adapters or stealthy backdoor injections.
- **Robustness:** Investigating adversarial prompts designed to fool the fingerprint, and developing defences.
- **Efficiency:** Reducing the number of required probes without degrading accuracy, to make LIIH practical for API-rate-limited deployments.
- **Reference-free extension:** The current framework requires a trusted reference set of benign models; a future extension could learn anomaly scores without explicit negative examples.

### 6.1 Limitations

While the LIIH framework demonstrates high diagnostic accuracy, several constraints regarding the experimental scale and environment must be acknowledged:

- **Dataset Granularity:** Our probe set relied on a relatively small sample of 100 data points from MMLU-Pro and 30 from TruthfulQA. In the pair-based setting, too few probes increase the risk that comparison vectors for structurally different modification types become indistinguishable — for example, a fine-tuned pair and a backdoored pair producing near-

identical LIIH signatures — which would cause the classifier to conflate modification types rather than separate legitimate from tampered pairs.

- **Sample Size:** Our experiment comprises 38 model pairs (28 train / 10 test). The 100% test accuracy achieved should be interpreted as a proof-of-concept; a larger, more balanced corpus of pairs would be required to establish a statistically robust confidence interval for the framework’s performance in production environments.
- **Homogeneous Hardware Environment:** All models were evaluated on identical hardware under the same conditions. As a result, the component-level ablation shows that no single component produces a unique, irreplaceable test signal on this corpus — each is fully substitutable by the remaining three. In particular, Component T (Temporal) is designed to detect infrastructure-level modifications such as quantization or hardware substitution, signals that only manifest on heterogeneous deployments. Evaluating LIIH across mixed hardware configurations, and on a larger pair corpus where inter-component redundancy cannot be assumed, remains an important direction for future work.

## References

- Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, et al. SmolLM2: When smol goes big — data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, 2023.
- Sid Black, Gao Leo, Phil Wang, Connor Leahy, and Stella Biderman. Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow, March 2021. URL <https://doi.org/10.5281/zenodo.5297715>.
- Lingjiao Chen, Matei Zaharia, and James Zou. How is chatgpt’s behavior changing over time? *Harvard Data Science Review*, 6(2), 2024.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Dirk Groeneveld, Iz Beltagy, Evan Pete Walsh, et al. OLMo: Accelerating the science of language models. *arXiv preprint arXiv:2402.00838*, 2024.
- Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.
- Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, et al. OPT-IML: Scaling language model instruction meta learning through the lens of generalization. *arXiv preprint arXiv:2212.12017*, 2022.
- Shiyao Li, Xuefei Ning, Luning Wang, Tengxuan Liu, Xiangsheng Shi, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Evaluating quantized large language models. *arXiv preprint arXiv:2402.18158*, 2024.
- Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- Dario Pasquini, Evgenios M. Kornaropoulos, and Giuseppe Ateniese. Llmmap: Fingerprinting for large language models, 2025. URL <https://arxiv.org/abs/2407.15847>.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 2019.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

- Teven Le Scao, Angela Fan, Christopher Akiki, et al. BLOOM: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.
- Shuo Shao, Yiming Li, Hongwei Yao, Yifei Chen, Yuchen Yang, and Zhan Qin. Reading between the lines: Towards reliable black-box llm fingerprinting via zeroth-order gradient estimation. *arXiv preprint arXiv:2510.06605*, 2025.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shengbang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.